

Speech Module - Technical Documentation

Mateusz Block, Lucjan Butko, Adam Macko

March 2026

Contents

1	Project Overview and Purpose	2
2	System Architecture	2
3	Environment and Dependencies	2
3.1	Required Libraries	2
3.2	AI Models	3
4	Core Modules Implementation	3
4.1	NLP Pipeline	3
4.2	Intent Detection	5
4.3	Weather Integration	6
4.4	Database Architecture	6
4.5	Algorithms for Database Retrieval	7
4.6	Text-to-Speech and Asynchronous Streaming	7
4.7	Identity and Emotion Management	8
5	Prompt Engineering	8
5.1	System Prompt	8
5.2	Deterministic Retrieval and Tool Calling	8
5.3	Conversation History and Context Buffer	9

1 Project Overview and Purpose

The main goal of the speech module, developed as part of the "humanoid-head" project, is to enable the robot to conduct natural and meaningful conversations with humans. Our team's task is to ensure that the machine not only reacts to rigid commands but can also respond fluently and logically to questions, mimicking a real conversation.

The assistant was designed with the students of the Gdańsk University of Technology in mind. Its practical applications within this interaction include:

- Providing precise navigation directions, making it easier to reach specific rooms in university buildings (such as EA or NE).
- Providing up-to-date information about weather conditions.
- Engaging in casual conversations on general topics, using artificial intelligence to generate natural-sounding responses.

2 System Architecture

The system is designed as a continuous loop that runs until it is stopped manually. It uses a pipeline structure with three main steps:

- **Speech-to-Text (STT):** The system uses a microphone to listen to the user. It records the sound and uses the Whisper AI model (the "small" version) to change the user's voice into text.
- **Intent Detection and Processing (LLM):** After getting the text, the system checks what the user wants using the `IntentDetector`.
 - If the user asks about the weather ("POGODA") or university classes ("PG"), the system uses specific logic to find this data.
 - For all other questions, it sends the text to a Large Language Model. The project uses the Ollama framework with the "gemma3:4b" model, connected through the LangChain library. The model gets a special prompt with a database of rooms and weather data to know how to answer correctly.
- **Text-to-Speech (TTS):** When the text answer is ready, the system uses the `pyttsx3` library to read it out loud.

3 Environment and Dependencies

3.1 Required Libraries

The project relies on a variety of Python libraries for audio processing, web scraping, system integration, and data manipulation. The key dependencies are described below:

- **BeautifulSoup4** (`beautifulsoup4`): Used for parsing HTML documents. It is utilized to scrape information and extract staff details from the university's websites.
- **LangChain** (`langchain-core`, `langchain-ollama`): Manages the integration with local Large Language Models via Ollama and formats chat prompt templates.
- **PyAudio** (`pyaudio`): Provides Python bindings for PortAudio, essential for capturing live audio streams from the microphone.

- **Python Weather** (`python-weather`): An asynchronous library used to fetch current weather data and conditions.
- **pyttsx3** (`pyttsx3`): An offline Text-to-Speech (TTS) conversion library that allows the assistant to vocalize its responses to the user.
- **RapidFuzz** (`rapidfuzz`): A fast string matching library used for fuzzy search. It helps in matching spoken teacher names against the database, accommodating slight mispronunciations or transcription errors.
- **Requests** (`requests`): An HTTP library utilized to send requests to external university web pages for data scraping purposes.
- **SpeechRecognition** (`speechrecognition`): A wrapper library that facilitates capturing microphone input, detecting speech, and automatically adjusting for ambient noise levels.

3.2 AI Models

The system implements the following artificial intelligence models and their associated frameworks to process user input, recognize entities, and generate appropriate responses:

- **Whisper ("small")**: A Speech-to-Text (STT) model developed by OpenAI (managed via the `openai-whisper` library), used to transcribe the audio captured from the user's microphone into text.
- **Gemma 3** (`gemma3:4b`): A Large Language Model (LLM) served via Ollama. It is responsible for generating conversational responses for general queries when no specific predefined intent is detected.
- **GLiNER** (`urchade/gliner_multi-v2.1`): A Named Entity Recognition (NER) model (implemented via the `gliner` library) utilized to extract specific information from the transcribed text. It identifies entities such as room codes and person names (using `room code` and `person` labels) to assist in university navigation queries.
- **spaCy** (`pl_core_news_sm`): A pre-trained Polish language model operating within the `spacy` NLP library. It performs word lemmatization and tokenization to quickly and accurately match user queries against predefined intent keywords.

4 Core Modules Implementation

4.1 NLP Pipeline

The Natural Language Processing (NLP) pipeline acts as the central coordinator of the humanoid head's interaction system. Implemented primarily within the `NlpModel` class, it manages the continuous loop of listening, understanding, and responding to the user. The pipeline is composed of three main stages:

1. **Speech-to-Text (STT)**: The pipeline continuously captures ambient audio via the microphone using the `SpeechRecognition` library, automatically adjusting for background noise. Once speech is detected, the audio is processed by the Whisper STT model to generate an accurate text transcription.
2. **Intent Routing & Processing**: The transcribed text is immediately forwarded to the Intent Detection module. This module classifies the user's query into specific operational branches:

- If a **Weather Intent** is detected, the system bypasses the LLM and directly queries the weather integration module to retrieve the current conditions.
 - If a **PG (Politechnika Gdańska) Intent** is detected, the text is preprocessed and analyzed by the GLiNER model to extract entities such as room codes or personnel names, which are then passed to the university database modules.
 - If **No Specific Intent** (General) is detected, the transcribed text is fed into the locally hosted Large Language Model (Gemma 3) via LangChain to generate a natural, conversational response.
3. **Text-to-Speech (TTS):** Regardless of which branch processes the query, the resulting answer is sent to the TTS module (`pyttsx3`) to be synthesized into audible speech, completing the interaction cycle.

To better illustrate the data flow within the `NlpModel`, Figure 1 presents the architectural diagram of the pipeline.

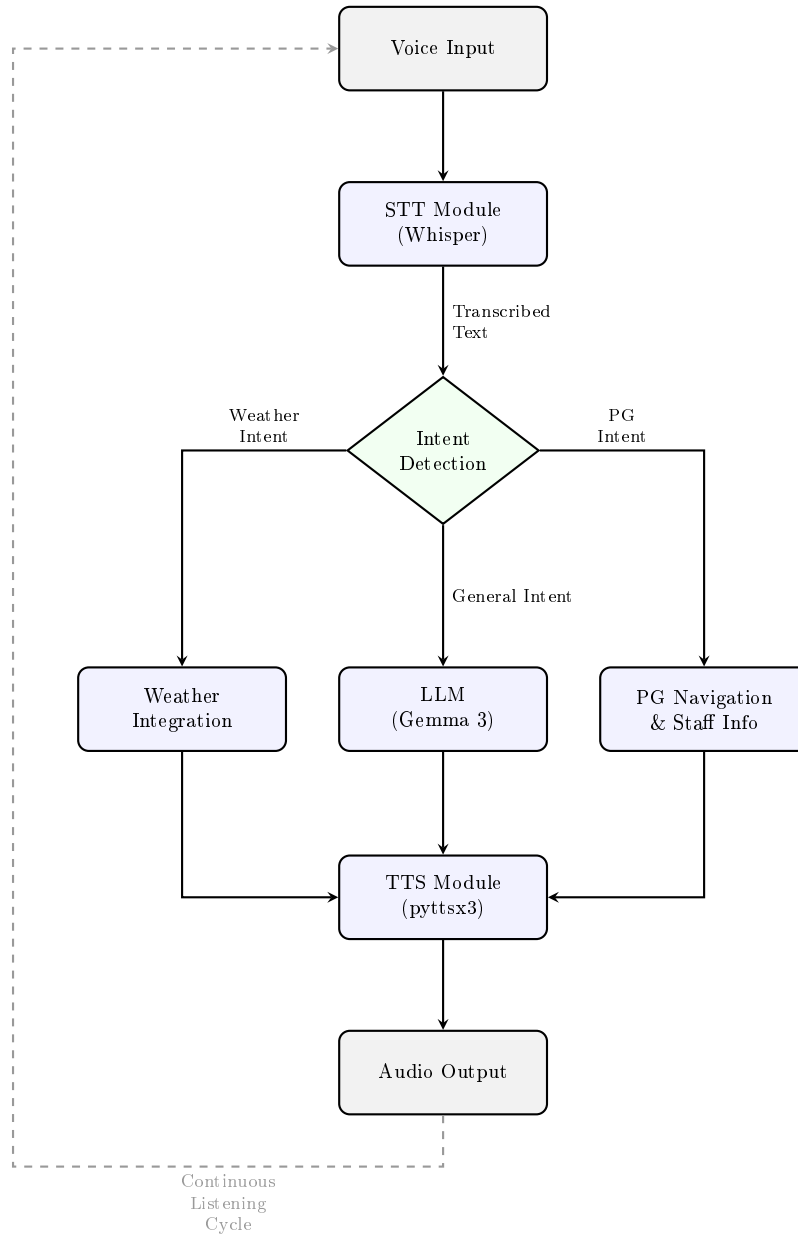


Figure 1: NLP Pipeline Data Flow.

4.2 Intent Detection

The intent detection module is responsible for classifying the user’s transcribed speech into one of the predefined categories, which determines how the query is routed within the NLP pipeline. This logic is encapsulated within the `IntentDetector` class.

Initialization and Keyword Sets

The module uses the `spacy` library along with the Polish language model (`pl_core_news_sm`) to perform advanced text analysis. During initialization, it defines two primary sets of base words used

for classification:

- **PG Keywords:** Terms related to the university, such as "sala", "wykład", "harmonogram", "profesor" and "dojść".
- **Weather Keywords:** Terms related to meteorological conditions, such as "pogoda", "temperatura", "deszcz" and "wiatr".

Operating Principle

The core classification is handled by the `detect_intent()` method, which processes the input through the following steps:

1. **Text Preprocessing:** The raw input string is converted to lowercase and passed through the spaCy processing pipeline.
2. **Lemmatization and Filtering:** The module iterates over the generated tokens, actively filtering out punctuation marks (`is_punct`) and common Polish stop words (`is_stop`). It extracts the base form (lemma) of each remaining token and stores them in a set for optimized searching.
3. **Set Intersection and Classification:** The system uses logical set intersections to check for overlaps between the extracted lemmas and the predefined keyword sets.
 - If an intersection with the university keywords is found, it returns the "PG" intent.
 - If an intersection with the weather keywords is found, it returns the "POGODA" intent.
 - If there are no matches, the function returns "OGOLNA" intent (general intent), signaling the pipeline to process the query using the large language model.

4.3 Weather Integration

4.4 Database Architecture

To ensure deterministic, high-performance retrieval of spatial and personnel data, the speech module decouples its operational knowledge from the core Large Language Model. Instead of relying on the parametric memory of the LLM—which is prone to hallucinations and temporal degradation—a structured data storage layer was engineered. This layer consists of specialized databases formatted in JSON. The choice of JSON provides a lightweight, readable, and highly structured schema that allows for seamless parsing, and straightforward keyword-based queries within the Python environment.

The storage layer is fundamentally divided into three distinct database files, each handling a specific semantic domain of the Faculty of Electronics, Telecommunications and Informatics (WETI):

- **Room Navigation Database (`room_directions.json`):** This database establishes a structural mapping of numbered classrooms, laboratories, and administrative offices across the EA (WETI A) and NE (WETI B) buildings. Each entry is keyed by a unique identifier combining the building code and the room number. The underlying payload contains explicit, step-by-step human-readable navigational routing instructions. These directions are written from the perspective of a user.
- **Specialized Locations Database (`db_specialrooms.json`):** To handle facility locations that do not follow standard room-numbering conventions, a separate semantic database was implemented. This file stores routing directions to these facilities.
- **Academic Staff Database (`teachers_info.json`):** This repository acts as the primary registry for the faculty's human resources and academic staff. Every entry is mapped to the full name of the researcher or administrative employee. The schema encapsulates critical metadata

fields, including formal academic titles, the formally assigned institutional department (chair), and the office (building and room number). In instances where a staff member does not have a permanently assigned public office, the room schema dynamically defaults to a standardized "Brak" token, preventing the system from outputting corrupted or missing structural data.

By isolating these data vectors into a flat, well-indexed JSON topography, the system achieves sub-millisecond data lookup times. This architecture guarantees that any navigational or informational response delivered is grounded strictly in verified data.

4.5 Algorithms for Database Retrieval

The core operational logic of the retrieval layer is driven by a suite of dedicated Python scripts designed specifically to query the structured JSON databases. They receive data that has already been extracted into a concrete, specific format by the preceding GLiNER sequence. The scripts process these inputs to return the corresponding, factual data.

- **Room Localization (`find_room.py`):** This script is tasked with querying the room navigation database. Before a database query is executed, the raw string extracted from user input is subjected to a normalization pipeline that eliminates whitespace discrepancies, converts characters to uppercase, and applies regular expressions to standardize the string. If the normalized key exists within `room_directions.json`, the script directly returns the explicit human-readable navigation payload.
- **Specialized Locations Retrieval (`find_aud.py`):** Operating analogously to the room localization script, this component handles queries regarding facility areas that do not conform to standard, sequential room numbers (such as auditoriums, libraries, or cafeterias). By mirroring the parsing structure of the room lookup but targeting `db_specialrooms.json`, it isolates semantic location queries and rapidly routes the corresponding floor parameters and customized directions.
- **Academic Staff Query and Coordination Pipeline (`find_teacher.py`):** This script manages the retrieval of personnel data from `teachers_info.json`. When a staff identity is queried, the script executes a fuzzy string matching search across the database keys. Upon finding a valid entry, it extracts the staff member's profile metadata and returns their assigned coordinates (building prefix and office number). To ensure seamless operational continuity, these coordinates are then programmatically passed into the `find_room.py` pipeline, which automatically resolves and streams the explicit routing directions for that specific office to the user.

Interactive Fallback for Ambiguous Queries

To further improve the reliability of the staff retrieval algorithm, an interactive fallback mechanism was introduced. When the fuzzy matching algorithm identifies multiple potential candidates with a confidence score falling between the strict top threshold and the baseline threshold, the system actively engages the user. It presents the top three most probable faculty members and asks the user to select the correct one by speaking a corresponding number (0, 1, or 2). This interactive disambiguation ensures accurate guidance even when the user's initial query is vague or the STT transcription is imprecise.

4.6 Text-to-Speech and Asynchronous Streaming

To improve the naturalness and responsiveness of the conversation, the system transitioned from the traditional `pyttsx3` library to **PiperTTS**, utilizing a high-quality Polish acoustic model in `.onnx` format. This upgrade significantly enhances the voice quality and makes the humanoid sound much more lifelike.

Furthermore, to minimize the perceived latency, the TTS module implements asynchronous streaming. Instead of waiting for the Large Language Model to generate the entire response, the system utilizes Python threading and queues. A dedicated background thread monitors the LLM's output character by character. Upon detecting punctuation marks (indicating the end of a phrase or sentence), it immediately chunks the text and sends it to the PiperTTS engine. This architecture allows the humanoid head to begin speaking almost instantly while the LLM continues generating the rest of the response in parallel.

4.7 Identity and Emotion Management

A key advancement in creating a compelling human-robot interaction is the ability to recognize the user's identity and emotional state. The system features a dedicated `get_new_user_name` routine, where the assistant introduces itself and asks for the user's name if they are unrecognized. The transcribed spoken name is then matched against a local database of Polish names (`imiona_polskie.txt`) using fuzzy matching via the `rapidfuzz` library. This ensures robust name recognition even if the Speech-to-Text model slightly misinterprets the pronunciation.

In addition to voice interactions, the speech module integrates a local API server built with the **Flask** framework (exposing the `/api/data` endpoint). This endpoint allows external systems (such as a vision or facial recognition module) to continuously update the current user's identity and detected emotion. The emotions are mapped through an `EMOTION_PL_MAP` to Polish equivalents (e.g., mapping "Happy" to "radość"). This state is subsequently synchronized with the main Raspberry Pi controller via HTTP requests to maintain a coherent behavioral state across all of the robot's subsystems.

5 Prompt Engineering

5.1 System Prompt

The system prompt establishes the assistant as a helpful guide for students of the Gdańsk University of Technology.

If the intent module detects a general conversational intent rather than a specific command, the system uses this prompt to guide the Large Language Model. The user's transcribed query is simply injected into the `{question}` placeholder, allowing the AI to generate a natural, contextual response.

5.2 Deterministic Retrieval and Tool Calling

During the initial development phases, the implementation of a Retrieval-Augmented Generation (RAG) architecture was considered. Ultimately, the RAG approach was abandoned in favor of a deterministic "Tool Calling" architecture.

When the system detects a specific, university-related intent, it bypasses the LLM's generative capabilities. Instead, the text is processed by the **GLiNER** model (for Named Entity Recognition) to extract key parameters, which then triggers dedicated Python scripts to fetch hardcoded, verified data. The two primary retrieval tools used in this process are:

- **find_room.py (Exact Match Retrieval):** This script is executed when the GLiNER model extracts a `room code` entity. It takes a formatted input string, splits it into building and room components using a comma delimiter, and performs a strict exact-match search within the `room_directions.json` database. If a match is found, it directly returns the precise, step-by-step navigation instructions.
- **find_teacher.py (Fuzzy Match Retrieval):** This script is invoked when a `person` entity is detected by GLiNER. Because Speech-to-Text models often mistranscribe proper nouns and surnames, an exact match approach would frequently fail. To solve this, the script utilizes

the `rapidfuzz` library to calculate a string similarity score against the `teachers_info.json` database. If the match score meets or exceeds the defined threshold of 50%, the system assumes a successful identification and retrieves the professor’s assigned office location.

By employing these retrieval scripts as external tools, the system guarantees factual accuracy in its navigational responses and significantly reduces latency, completely mitigating the risk of LLM hallucinations.

5.3 Conversation History and Context Buffer

To support multi-turn conversations and maintain context over time, a conversation history buffer was introduced. Implemented using a double-ended queue (`collections.deque`), it retains a fixed number of recent interactions, defined by the `CONVERSATION_BUFFER_LEN` constant. When the buffer reaches its capacity, the oldest exchanges are automatically discarded to keep the LLM’s context window optimized and prevent memory overflow.

The system prompt has been enhanced to inject dynamic variables directly into the template before invoking the LLM. The pipeline provides the structured `{history}` of the conversation, the current user’s `{name}`, and their detected `{emotion}`. This context injection allows the Gemma model to highly personalize its responses—for instance, by addressing the user by their name or adjusting its conversational tone based on whether the user is currently happy, sad, or neutral.